

Revisão para a A1 — Roteiro de estudo

Curso: Bacharelado em Sistemas de Informação — 2º período **Disciplina:** Banco de Dados I
Professor: Me. Alysson Martins Bruno **Conteúdo revisado:** Aulas 01 a 06 **Prova A1:** 10 de outubro de 2026, sábado, período vespertino

Como usar este roteiro

Este roteiro leva você, passo a passo, por tudo o que cai na prova A1. Ele está dividido em **etapas numeradas**. Cada etapa tem um tempo estimado e segue sempre a mesma ordem:

1. **Resumo** — o essencial do tema, em poucas linhas;
2. **Exemplo resolvido** — um caso concreto, com a solução comentada;
3. **Armadilhas** — os erros que mais aparecem nas respostas;
4. **Perguntas de fixação** — tente responder no caderno **antes** de ler a resposta;
5. **Confira a resposta** — a explicação, logo abaixo.

O tempo total é de **cerca de 3 horas e 20 minutos**. Não precisa ser de uma vez: faça uma ou duas etapas por dia, na ordem. A Etapa 10 é o **simulado**, que deve ser feito sem consultar nada. Depois dele vem a **Resolução comentada do simulado**, para você se corrigir sozinho.

Etapa	Tema	Tempo
0	Mapa da prova	10 min
1	Dado, informação, conhecimento, banco de dados e SGBD	15 min
2	SGBD relacional, NoSQL ou arquivo comum?	15 min
3	Níveis de abstração e independência de dados	15 min
4	Modelo relacional: tupla, atributo, domínio e chaves	15 min
5	DER, cardinalidade e notação pé de galinha	15 min
6	Álgebra relacional: seleção (σ) e projeção (π)	15 min
7	Arquitetura cliente/servidor do MySQL	15 min
8	Processo de modelagem: da história ao modelo lógico	15 min
9	DDL no MySQL: tipos, restrições e ordem de criação	30 min
10	Simulado (sem consulta)	40 min

Tenha à mão caderno, lápis e caneta. Na prova tudo é escrito à mão, então **treine escrevendo**.

Etapa 0 — Mapa da prova (10 min)

Como é a prova

Item	Como é
Data	10/10/2026 , sábado, período vespertino
Forma	individual, sem consulta , escrita em papel , com caneta
Parte I	10 questões objetivas , de 0,5 ponto cada (5,0 pontos), com alternativas A, B, C e D
Parte II	1 questão discursiva , de 5,0 pontos
Total	10,0 pontos

As objetivas seguem o estilo ENADE, o mesmo das listas de exercícios das aulas:

- **afirmações I, II, III e IV** — você marca quais estão corretas;
- **asserção e razão** — duas frases ligadas por PORQUE; você decide se cada uma é verdadeira e se a segunda explica a primeira;
- **situação-problema** — um caso, uma tabela ou um trecho de SQL, e você escolhe a análise correta.

A discursiva vale **metade da prova**. Ela traz um pequeno caso de negócio para você modelar e transformar em tabelas no MySQL — exatamente o que as Etapas 5, 8 e 9 treinam.

O que cai e o que não cai

Cai (Aulas 01 a 06)	Não cai
dado, informação e conhecimento; papel do SGBD	INSERT, SELECT, UPDATE e DELETE como comandos a escrever
SGBD relacional, NoSQL e arquivo comum	formas normais (normalização)
níveis de abstração e independência de dados	JOIN e consultas com várias tabelas
modelo relacional: tupla, atributo, domínio, chave primária, chave natural, chave estrangeira, integridade referencial	ferramentas usadas na prática (ambientes de laboratório, sites de SQL)
DER com cardinalidade e notação pé de galinha	
álgebra relacional: seleção (σ) e projeção (π)	
arquitetura cliente/servidor do MySQL	
processo de modelagem: fronteira, substantivos e verbos, conceitual $\rightarrow$ lógico $\rightarrow$ físico	
DDL no MySQL: tipos, NOT NULL, DEFAULT, UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY, ON DELETE e ON UPDATE, ordem de criação, prever o erro de um script	

Autoavaliação: eu sei...

Leia cada frase e marque **sim** ou **não**. Cada “não” diz para qual etapa você precisa dar mais atenção.

Eu sei...	Se não, vá para
separar dado, informação e conhecimento em um exemplo	Etapa 1
explicar por que redundância causa inconsistência	Etapa 1
dizer quando usar SGBD relacional, NoSQL ou um arquivo comum	Etapa 2
classificar um artefato como conceitual, lógico ou físico	Etapa 3
contar tuplas e atributos e dizer se uma coluna serve de chave primária	Etapa 4
explicar a diferença entre chave primária, chave natural e chave estrangeira	Etapa 4
descobrir a cardinalidade perguntando nos dois sentidos	Etapa 5
desenhar e ler as pontas de uma linha no pé de galinha	Etapa 5
calcular o resultado de π e σ sobre uma tabela pequena	Etapa 6
dizer o que fazem mysqld, mysql, camada de conexão, camada SQL e InnoDB	Etapa 7
decidir em que tabela fica a chave estrangeira de um 1:N	Etapa 8
escolher o tipo certo para dinheiro, código e data	Etapa 9
escrever um CREATE TABLE com PK, FK, UNIQUE, CHECK e DEFAULT	Etapa 9
prever o erro que o MySQL dá para um script em ordem errada	Etapa 9

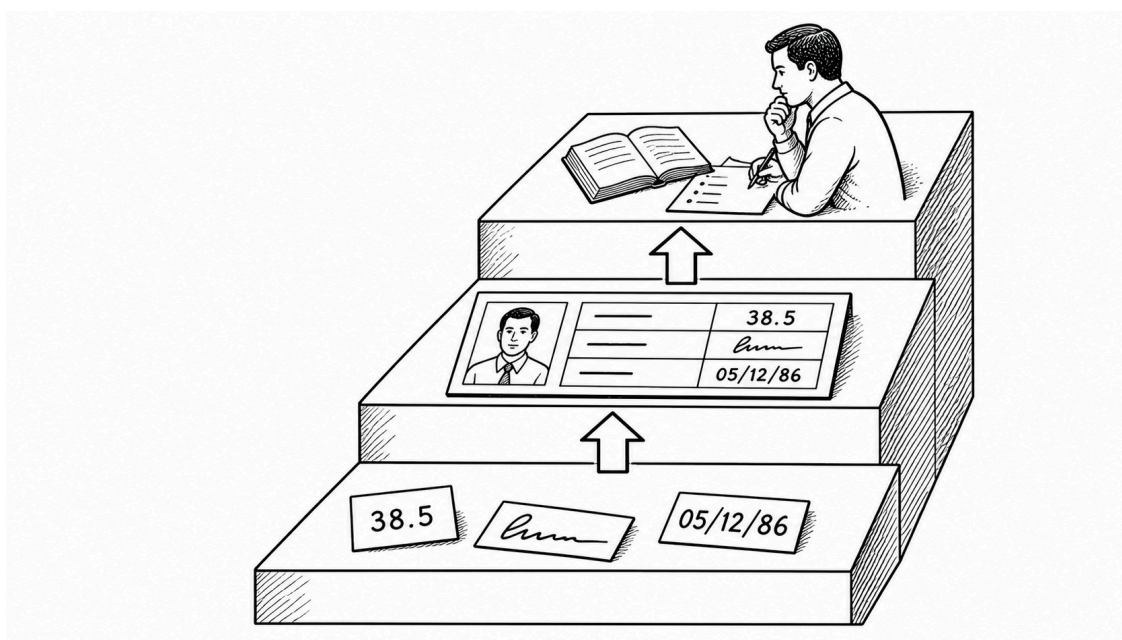
Mesmo que você tenha marcado “sim” em tudo, faça as perguntas de fixação de cada etapa. Elas mostram se o “sim” aguenta uma pergunta difícil.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 1 — Dado, informação, conhecimento, banco de dados e SGBD (15 min)

Resumo

Nível	O que é	Pergunta que responde
Dado	representação registrada, sem contexto	<i>O que foi registrado?</i>
Informação	dado com contexto e significado	<i>O que isso quer dizer?</i>
Conhecimento	informação combinada com experiência ou regra, que permite decidir	<i>O que eu faço?</i>



Da base para o topo: dados soltos, a ficha que dá contexto e a pessoa que decide

- **Banco de dados** é uma coleção de dados relacionados e organizados. O **SGBD** (Sistema Gerenciador de Banco de Dados) é o conjunto de programas que gerencia esse banco. Resumo: o *banco guarda*; o *SGBD gerencia*. MySQL, PostgreSQL e Oracle são SGBDs.
- O SGBD existe para: controlar a **redundância**, facilitar o **acesso**, evitar problemas de **concorrência** (duas pessoas ao mesmo tempo), impor **integridade** (regras), garantir **atomicidade** (tudo ou nada) e apoiar a **segurança**.
- As anomalias do controle por planilha ou arquivo:

Anomalia	Exemplo
Redundância	o mesmo cliente cadastrado em três linhas
Inconsistência	as três linhas com telefones diferentes
Concorrência	duas pessoas salvam o arquivo ao mesmo tempo e o trabalho de uma se perde
Perda de integridade	uma venda registrada para um cliente que não existe
Falta de segurança	quem abre o arquivo vê e altera tudo

Redundância é a causa; inconsistência é a consequência. Se o dado estivesse em um lugar só, não haveria duas versões para discordar.

Exemplo resolvido

Um clube de corrida de rua de Arraias guarda as inscrições em uma planilha. O valor 52 aparece em uma célula. No relatório, lê-se: “*O corredor Tiago completou os 10 km em 52 minutos, na prova de domingo*”. Como o clube combina que quem faz menos de 55 minutos entra na equipe principal, o técnico chamou Tiago para o grupo de treino avançado.

- 52 sozinho → **dado** (pode ser idade, peso, preço...).
- A frase do relatório → **informação** (diz quem, o quê, quando e em que unidade).
- A decisão de chamar Tiago, usando a regra do clube → **conhecimento** aplicado.

O clube também percebeu que Tiago está cadastrado duas vezes, com e-mails diferentes. O cadastro duplicado é a **redundância**; os dois e-mails que não batem são a **inconsistência**.

Armadilhas

- Achar que o **sistema** “toma a decisão” e por isso a decisão é um dado. A decisão usa informação mais uma regra ou experiência: é conhecimento.
- Inverter causa e consequência: não é a inconsistência que causa a redundância.
- Achar que o SGBD “entende” o mundo. Ele só sabe o que alguém registrou, sob a regra que alguém definiu.

Perguntas de fixação

1. Em “Maria”, 12/05/2026 e “*Maria nasceu em 12/05/2026*”, o que é dado e o que é informação?
2. Duas atendentes abrem a mesma planilha, alteram linhas diferentes e salvam. As alterações de uma somem. Qual anomalia é essa, e qual finalidade do SGBD a evita?
3. Uma transferência tira R\$ 50,00 de uma conta e a energia cai antes de colocar na outra. Que propriedade do SGBD impede que o dinheiro suma?

Confira a resposta

1. “Maria” e 12/05/2026, soltos, são **dados**. A frase junta os dois com contexto (quem e o quê): é **informação**.
2. É **anomalia de concorrência**. O SGBD controla o acesso de vários usuários ao mesmo tempo, para que uma gravação não apague a outra.
3. **Atomicidade**: a transferência é uma transação; ou os dois passos acontecem, ou nenhum acontece.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 2 — SGBD relacional, NoSQL ou arquivo comum? (15 min)

Resumo

Situação	Escolha indicada
dados com estrutura fixa e regras claras (dinheiro, estoque, matrícula), usados por várias pessoas ao mesmo tempo	SGBD relacional (MySQL, PostgreSQL, Oracle)
registros cujos campos variam muito de um para outro, sem regras entre eles	pode favorecer NoSQL de documentos (MongoDB)
cache, contadores, filas	pode favorecer NoSQL chave-valor (Redis)
redes de relacionamento (quem conhece quem)	pode favorecer NoSQL de grafos (Neo4j)
dados usados por uma pessoa só, uma vez, sem regras, que não precisam durar	arquivo comum (texto, planilha pessoal)

Regra prática: se os dados são **compartilhados**, **duram** no tempo e **obedecem a regras**, há motivo para usar um SGBD. Se, além disso, a estrutura é fixa e há dinheiro ou integridade envolvidos, o relacional é a escolha natural.

Exemplo resolvido

O clube de corrida tem três necessidades:

1. **Inscrições das provas.** Cada inscrição tem a mesma estrutura (corredor, prova, valor pago); o CPF não pode repetir; três voluntários registram inscrições ao mesmo tempo no dia da prova; o dinheiro não pode sumir. → **SGBD relacional**.
2. **Leituras dos relógios de corrida.** Cada modelo de relógio envia campos diferentes (frequência cardíaca, altitude, cadência), e a lista de campos muda a cada modelo novo. Não há regra ligando uma leitura a outra. → pode favorecer **NoSQL de documentos**.
3. **Anotações pessoais do técnico** sobre os tênis que ele quer experimentar. Só ele usa, não há regra a proteger e ninguém mais precisa acessar. → **arquivo comum** basta.

Armadilhas

- Achar que NoSQL é “mais moderno, portanto melhor”. Não existe escolha superior em todos os casos: depende dos requisitos.
- Achar que planilha compartilhada resolve acesso simultâneo. É justamente a planilha que perde trabalho quando duas pessoas salvam juntas.
- Usar SGBD para tudo. Uma lista pessoal, usada uma vez, não precisa de servidor de banco.

Perguntas de fixação

1. Uma escola de idiomas precisa registrar matrículas e mensalidades pagas por centenas de alunos, com várias secretárias trabalhando ao mesmo tempo. Qual a escolha?
2. Uma estudante anota, só para ela, o horário das aulas da semana no bloco de notas do celular. Ela precisa de um SGBD?

Confira a resposta

1. **SGBD relacional:** estrutura fixa, dinheiro, regras (aluno existente, valor positivo) e acesso simultâneo.
2. **Não.** Os dados não são compartilhados com ninguém e não têm regras a proteger: um arquivo comum resolve.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 3 — Níveis de abstração e independência de dados (15 min)

Resumo

O projeto de um banco passa por **três níveis**, do mais abstrato ao mais concreto:

Nível	Pergunta	O que se produz	Depende de SGBD?
Conceitual	<i>O que guardar?</i>	DER: entidades (retângulos), relacionamentos (losangos), atributos e cardinalidades	não
Lógico	<i>Como organizar?</i>	tabelas, colunas, chave primária (PK) e chave estrangeira (FK)	não — serve para MySQL, PostgreSQL ou Oracle
Físico	<i>Como armazenar?</i>	CREATE TABLE com os tipos do produto (VARCHAR, DECIMAL...), índices, motor	sim

- O conceitual é o nível que se mostra ao cliente para **validar as regras do negócio**.
- **Independência de dados:** o administrador pode trocar o disco, criar um índice ou mudar o formato interno de armazenamento **sem reescrever os programas**. A aplicação continua pedindo os dados do mesmo jeito, e o SGBD continua respondendo.

Exemplo resolvido

O clube de corrida produziu três documentos:

1. um desenho com os retângulos CORREDOR, PROVA e INSCRIÇÃO, com losangos e cardinalidades, sem nenhum tipo de coluna, mostrado à diretoria do clube → **conceitual**;
2. `inscricao (id PK, corredor_id FK, prova_id FK, numero_peito, valor_pago)`, sem tipos → **lógico**;
3. `CREATE TABLE inscricao (... valor_pago DECIMAL(8,2) NOT NULL ...)` → **físico**.

Mais tarde, o servidor do clube ganhou um disco maior e um índice novo. O sistema de inscrições continuou funcionando sem nenhuma alteração no código: isso é **independência de dados**.

Armadilhas

- Achar que o lógico já é MySQL. Se não há tipo do produto nem comando, ainda é lógico.
- Achar que o DER “é só rascunho”. Ele é o modelo conceitual, e é nele que o cliente confere as regras.
- Confundir independência de dados com “o SQL roda igual em qualquer SGBD”. Cada SGBD tem seu dialeto; independência é a aplicação não depender de como o dado é **guardado**.

Perguntas de fixação

1. Um documento traz produto (id PK, nome, preco) e venda (id PK, produto_id FK, data), sem tipos. Em que nível ele está?
2. Um comando traz preco DECIMAL(10,2) NOT NULL. Em que nível ele está? Por quê?

Confira a resposta

1. **Lógico**: já tem tabelas, PK e FK, mas não depende de um SGBD específico.
2. **Físico**: usa um tipo do MySQL (DECIMAL(10,2)) e uma restrição escrita em SQL; é a implementação no produto.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 4 — Modelo relacional: tupla, atributo, domínio e chaves (15 min)

Resumo

Termo formal	Termo usual
Relação	tabela
Tupla	linha, registro
Atributo	coluna, campo
Domínio	conjunto de valores permitidos para o atributo (o tipo)

RELAÇÃO: aluno

id	nome	idade
1	Ana	19
2	Carlos	20
3	Bruna	18

TUPLA (aponta para a linha 2)

ATRIBUTO (aponta para a coluna nome)

VALOR (aponta para o valor 19)

Anatomia de uma relação: tupla é a linha, atributo é a coluna

- A ordem das linhas e das colunas não importa; cada célula guarda **um único valor**.
- **Chave primária (PK)**: identifica cada tupla; **não repete e não aceita nulo**. Na disciplina, toda tabela recebe uma coluna id artificial.
- **Chave natural**: um atributo do mundo real que também não pode repetir (CPF, CRMV, ISBN, placa). Ela **convive** com o id: continua protegida por UNIQUE.
- **Chave estrangeira (FK)**: coluna que aponta para a chave de outra tabela.
- **Integridade referencial**: o valor da FK precisa existir na tabela referenciada. O SGBD recusa a linha “órfã”.

Exemplo resolvido

Tabela prova:

id	nome	percurso_km
1	Rústica 5K	5.0
2	Rústica 10K	10.0

Tabela inscricao (prova_id referencia prova.id):

id	corredor_id	prova_id	numero_peito
1	7	1	101
2	8	1	102
3	7	2	101

- inscricao tem **3 tuplas e 4 atributos**.

- prova_id **não** serve de chave primária: o valor 1 aparece duas vezes.
- numero_peito sozinho também não: 101 aparece duas vezes (em provas diferentes).
- Uma inscrição com prova_id = 5 seria **recusada**: não existe prova 5 (integridade referencial).

Armadilhas

- Contar a linha do cabeçalho como tupla. O cabeçalho são os nomes dos atributos.
- Achar que um atributo pode ser PK “porque toda linha tem valor”. PK exige **não repetir**, hoje e sempre — nomes repetem.
- Achar que a FK não pode repetir. Ela pode e costuma repetir: vários filhos apontam para o mesmo pai.
- Chamar de “chave candidata”. Na disciplina e na prova, o termo usado é **chave natural**.

Perguntas de fixação

1. Numa tabela cliente (id, nome, cpf), qual é a chave primária e qual é a chave natural?
2. Por que a FK prova_id pode repetir, e a PK id não?
3. Qual o domínio de uma coluna percurso_km DECIMAL(4,1)?

Confira a resposta

1. id é a chave primária (artificial); cpf é a chave natural, que recebe UNIQUE.
2. Porque várias inscrições podem ser da mesma prova (lado N do 1:N). Já o id identifica uma única inscrição.
3. Os números com até 4 dígitos, sendo 1 depois da vírgula (de -999,9 a 999,9); o CHECK pode limitar ainda mais, por exemplo, a valores maiores que zero.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 5 — DER, cardinalidade e notação pé de galinha (15 min)

Resumo

- **Entidade** (retângulo): algo com existência própria sobre o qual guardamos dados. Nome no singular.
- **Relacionamento** (losango ou linha nomeada): uma associação, que na fala do cliente aparece como **verbo**.
- **Cardinalidade**: quantas ocorrências de uma entidade se ligam a uma ocorrência da outra. Descubra **perguntando nos dois sentidos**:
 - “vários” e “um” → **1:N**;
 - “vários” e “vários” → **N:M** (no lógico, vira **tabela associativa**);
 - “um” e “um” → **1:1**.

Notação pé de galinha — o símbolo fica na ponta da linha, **junto da entidade** que ele descreve:

Símbolo	Significado
círculo	zero (opcional)
traço	um
pé de galinha	muitos

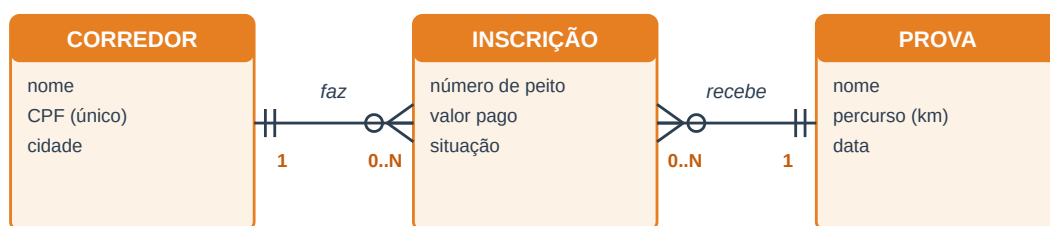
Na ponta de cada linha aparecem dois símbolos: o **mínimo** (círculo ou traço) e o **máximo** (traço ou pé de galinha).

Exemplo resolvido

Regras do clube: “*um corredor pode fazer várias inscrições, e cada inscrição é de um único corredor; uma prova recebe várias inscrições, e cada inscrição é de uma única prova*”.

- *Um corredor faz quantas inscrições?* Várias (ou nenhuma, se acabou de se cadastrar). *Uma inscrição é de quantos corredores?* Exatamente um. → **1:N**.
- Mesmo raciocínio para PROVA e INSCRIÇÃO → **1:N**.

DER — CORRIDA DE RUA (NOTAÇÃO PÉ DE GALINHA)

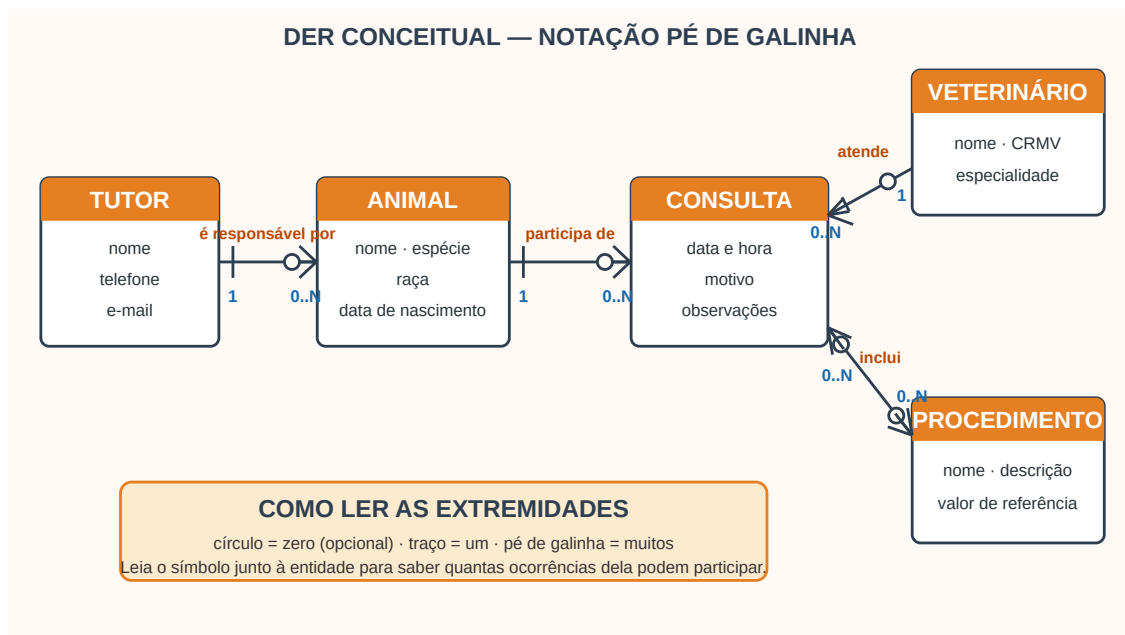


Um corredor faz 0..N inscrições; cada inscrição é de 1 corredor e de 1 prova.
círculo = zero (opcional) · traço = um · pé de galinha = muitos

DER da corrida na notação pé de galinha

Leitura da figura: junto de CORREDOR há **dois traços** (mínimo um, máximo um: toda inscrição tem exatamente um corredor). Junto de INSCRIÇÃO há **círculo e pé de galinha** (mínimo zero, máximo muitos: um corredor pode ter nenhuma ou várias inscrições).

O modelo da clínica veterinária, visto nas aulas, usa a mesma notação:



DER conceitual da clínica: tutor, animal, consulta, veterinário e procedimento

Armadilhas

- **Perguntar em um sentido só.** “Um professor dá várias disciplinas” não diz quantos professores cada disciplina pode ter.
- **Pôr o pé de galinha do lado errado.** O pé de galinha fica junto da entidade que aparece “muitas vezes” — no 1:N, junto do lado N.
- Transformar N:M em uma coluna. N:M sempre vira tabela associativa no modelo lógico.

Perguntas de fixação

1. “Cada ônibus faz várias viagens; cada viagem é feita por um único ônibus.” Qual a cardinalidade e onde fica o pé de galinha?
2. “Um curso tem várias disciplinas, e uma disciplina é oferecida em vários cursos.” Qual a cardinalidade e o que ela exige no modelo lógico?
3. “Cada funcionário tem um único crachá, e cada crachá é de um único funcionário.” Como ficam as pontas da linha?

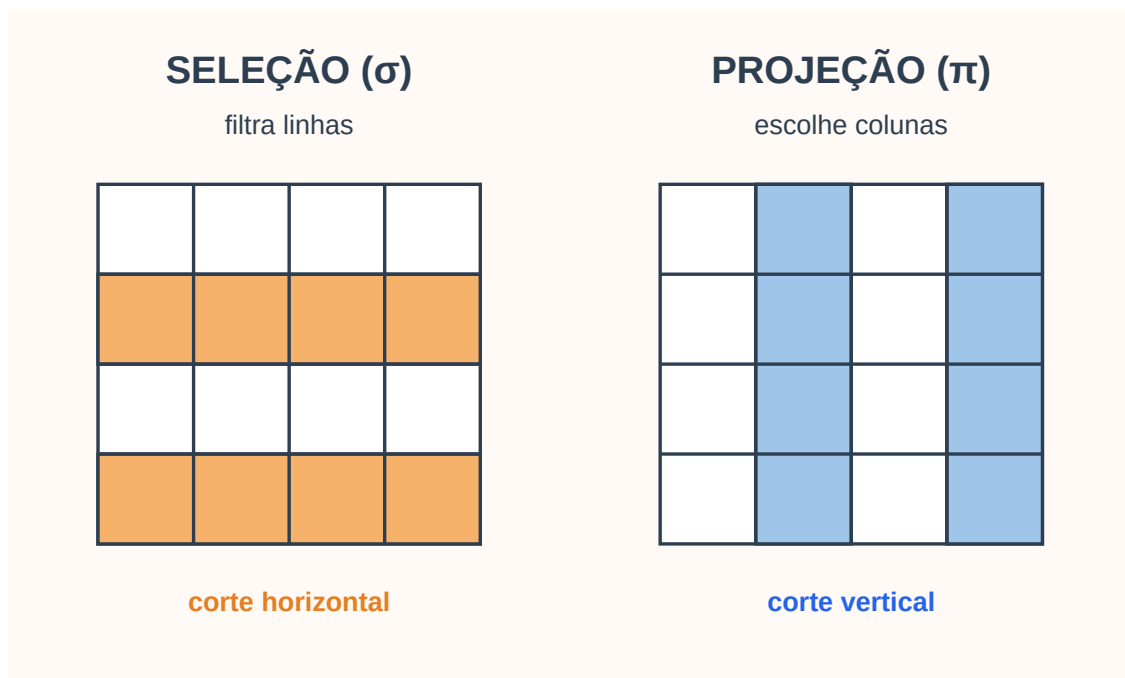
Confira a resposta

1. **1:N.** O pé de galinha fica junto de VIAGEM; junto de ÔNIBUS fica o traço.
2. **N:M.** Pé de galinha nas duas pontas; no lógico, uma tabela associativa (curso_disciplina), com uma FK para cada lado.
3. **1:1:** traço nas duas pontas (máximo um dos dois lados).

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 6 — Álgebra relacional: seleção (σ) e projeção (π) (15 min)

Resumo



Seleção corta linhas; projeção corta colunas

Operação	Símbolo	O que faz	Em SQL
Seleção	σ	filtra linhas que obedecem a uma condição; mantém todas as colunas	WHERE
Projeção	π	escolhe colunas ; mantém as linhas, eliminando as repetidas	lista de colunas do SELECT

- Toda operação devolve **uma nova relação**; por isso elas se encadeiam.
- Leia **de dentro para fora**: em $\pi_{\text{nome}}(\sigma_{\text{cidade} = \text{'Arraias'}}(\text{corredor}))$, primeiro σ filtra as linhas, depois π fica com a coluna nome.
- Como relação é um **conjunto**, a projeção não repete tuplas iguais.

Exemplo resolvido

Tabela corredor:

id	nome	cidade
1	Tiago	Arraias
2	Lúcia	Conceição do Tocantins
3	Rafael	Arraias
4	Tiago	Arraias

a) $\sigma_{\text{cidade} = \text{'Arraias'}}(\text{corredor}) \rightarrow$ linhas 1, 3 e 4, com as **3 colunas**.

b) $\pi_{\text{nome}}(\sigma_{\text{cidade} = \text{'Arraias'}}(\text{corredor})) \rightarrow$ primeiro as linhas 1, 3 e 4; depois só a coluna nome: Tiago, Rafael, Tiago. Como “Tiago” repete, a projeção devolve **1 coluna e 2 linhas**: Tiago e Rafael.

c) $\pi_{\text{cidade}}(\text{corredor}) \rightarrow$ **1 coluna e 2 linhas**: Arraias e Conceição do Tocantins.

Armadilhas

- Trocar os papéis: σ **não** escolhe colunas e π **não** filtra linhas.
- Ler de fora para dentro.
- Esquecer que a projeção elimina repetidos. (No SQL, o SELECT comum mantém repetidos; quem elimina é o SELECT DISTINCT. Na álgebra, vale a regra do conjunto.)

Perguntas de fixação

1. Com a tabela corredor acima, quantas linhas e colunas tem $\sigma_{\text{nome} = \text{'Tiago'}}(\text{corredor})$?
2. E $\pi_{\text{nome, cidade}}(\text{corredor})$?

Confira a resposta

1. **2 linhas** (ids 1 e 4) e **3 colunas** — a seleção mantém todas as colunas.
2. **2 colunas e 3 linhas**: (Tiago, Arraias), (Lúcia, Conceição do Tocantins) e (Rafael, Arraias). As linhas 1 e 4 ficam iguais depois da projeção e aparecem uma vez só.

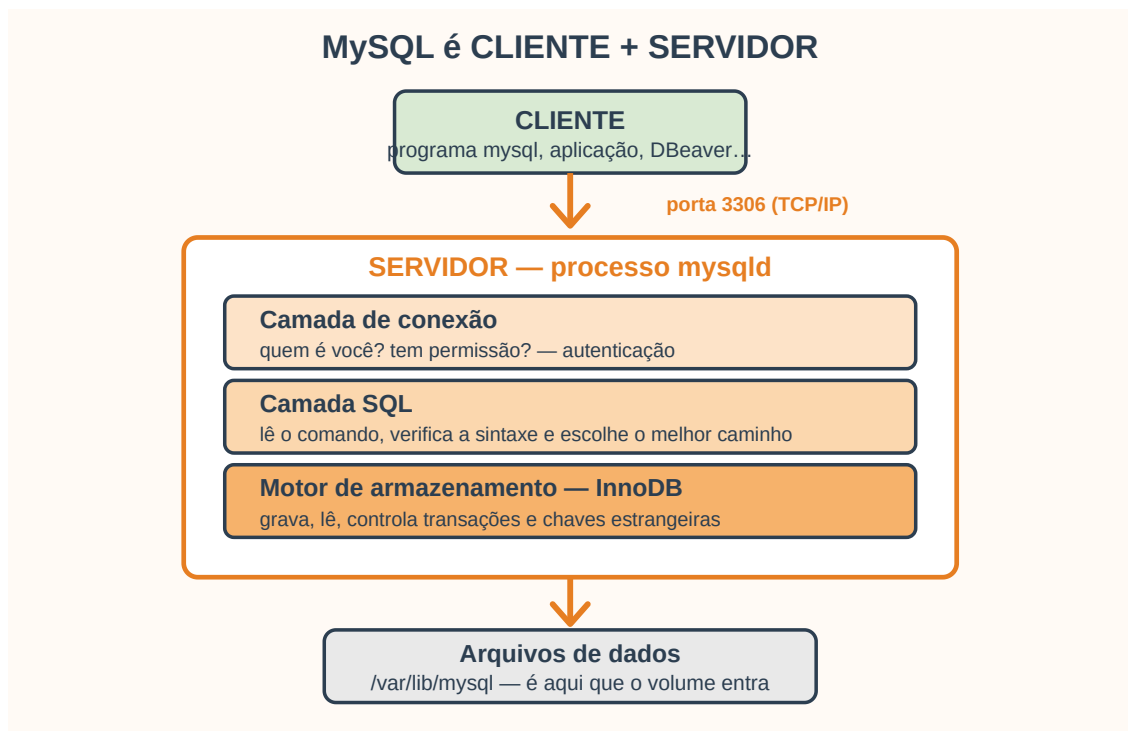
Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 7 — Arquitetura cliente/servidor do MySQL (15 min)

Resumo

O MySQL tem dois papéis:

- **servidor mysqld** — processo que fica rodando, guarda os dados, controla o acesso e responde (o d vem de *daemon*);
- **cliente mysql** — programa que envia os comandos ao servidor e mostra as respostas. O cliente **não abre os arquivos do banco**: ele pede, e o servidor responde. Fechar o cliente **não desliga** o servidor.



O MySQL por dentro: cliente, camadas do servidor e arquivos de dados

Camada do servidor	O que faz
Conexão	autentica o usuário e confere as permissões (é daqui que vem <code>Access denied</code>)
SQL	lê o comando, confere a sintaxe e escolhe o caminho mais eficiente
Motor de armazenamento	lê e grava de fato; o padrão é o InnoDB , que dá suporte a transações e chaves estrangeiras

Detalhes do cliente:

- todo comando termina em **ponto e vírgula**; sem ele, aparece `->` e o cliente espera o resto;
- `\c` abandona um comando começado errado;
- o servidor atende pela porta **3306**.

Bancos que aparecem no `SHOW DATABASES`; sem você ter criado — **internos, não se alteram à mão**:

Banco interno	Guarda
<code>information_schema</code>	metadados: nomes de tabelas, colunas e restrições
<code>mysql</code>	usuários e permissões
<code>performance_schema</code> e <code>sys</code>	dados de desempenho

Exemplo resolvido

Um voluntário do clube digitou um `CREATE TABLE` com uma vírgula sobrando antes do `)` e recebeu:

```
ERROR 1064 (42000): You have an error in your SQL syntax; ... near ')'
```

Quem produziu a mensagem? O **servidor**, na **camada SQL**, que confere a sintaxe. O cliente só exibiu a resposta. Como o comando foi recusado na análise, o motor de armazenamento nem chegou a ser acionado: nada foi criado.

Armadilhas

- Achar que o cliente mysql lê e grava os arquivos do banco.
- Achar que a camada SQL grava os dados. Quem grava é o **motor de armazenamento**.
- Achar que o -> é travamento. É o cliente esperando o ;.
- Achar que information_schema ou mysql são bancos “sobrando”, criados por engano.

Perguntas de fixação

1. Qual camada do servidor decide o caminho mais eficiente para executar um comando?
2. Qual componente garante que uma chave estrangeira seja respeitada ao gravar?
3. Depois de digitar SHOW TABLES e Enter, aparece ->. O que fazer?

Confira a resposta

1. A **camada SQL**, depois de conferir a sintaxe; a gravação fica com o motor de armazenamento.
2. O **motor de armazenamento InnoDB**, padrão do MySQL.
3. Digitar ; e Enter. Faltou o ponto e vírgula; o cliente estava esperando o fim do comando.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 8 — Processo de modelagem: da história ao modelo lógico (15 min)

Resumo

O caminho visto nas Aulas 05 e 06:

```
história → fronteira → substantivos → entidades e atributos
          → verbos → relacionamentos e cardinalidades → DER (conceitual)
          → tabelas, PK e FK (lógico) → CREATE TABLE (físico)
```

1. **Fronteira:** o que o sistema desta versão trata e o que fica de fora. Termo fora da fronteira não vira entidade agora.
2. **Substantivos** são **candidatos** a entidade ou atributo — não decisões. Pergunte: tem identidade própria? guardamos vários fatos sobre ele? participa de relacionamentos? Se só descreve outra coisa, é **atributo**. Se só dá contexto (o nome da empresa), não vira tabela.
3. **Verbos** sugerem **relacionamentos**. “Registrar” e “cadastrar” costumam ser funções do sistema, não relacionamentos.

4. Um **evento** com atributos próprios e histórico (consulta, inscrição, venda) vira **entidade**, e não só um losango.
5. Regras de conversão para o lógico:
 - cada entidade vira tabela, com coluna id como PK;
 - **em 1:N, a PK do lado 1 migra como FK para a tabela do lado N;**
 - N:M vira tabela associativa;
 - obrigatoriedade, unicidade e valores válidos viram restrições.

Exemplo resolvido

História: “O clube cadastra corredores com nome, CPF e cidade. As provas têm nome, percurso e data. O corredor se inscreve nas provas; cada inscrição tem número de peito, valor pago e situação. O clube também vende camisetas, mas isso fica para a próxima versão.”

Candidato	Decisão	Por quê
clube	contexto	há um único clube; não se guardam fatos sobre ele
corredor, prova	entidades	identidade própria e vários atributos
inscrição	entidade (evento)	tem atributos próprios e fica no histórico
nome, CPF, cidade, percurso, data, número de peito, valor, situação	atributos	descrevem outra coisa
camiseta	fora da fronteira	excluída desta versão

Modelo lógico:

```

CORREDOR (id PK, nome, cpf UQ, cidade)
PROVA (id PK, nome, percurso_km, data_prova)
INSCRICAO (id PK, corredor_id FK, prova_id FK, numero_peito,
           valor_pago, situacao)
  
```

As duas FKs ficam em inscricao, porque ela é o lado N dos dois relacionamentos 1:N.

Armadilhas

- Transformar todo substantivo em tabela (“clube”, “data”, “camiseta”).
- **FK no lado errado.** Pôr inscricao_id dentro de corredor exigiria uma coluna por inscrição — não funciona. Cada linha do lado N aponta para **uma** linha do lado 1.
- Usar nome como chave primária: nomes repetem e mudam.

Perguntas de fixação

1. “Cada setor tem vários funcionários; cada funcionário trabalha em um setor.” Em que tabela fica a FK?
2. Em um sistema de oficina de costura, “a oficina” deve virar tabela?

Confira a resposta

1. Em funcionario, como setor_id — o funcionário é o lado N.
2. Não, se o sistema atende uma única oficina: o termo só dá contexto. Viraria entidade se o sistema tivesse de guardar dados de várias oficinas.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 9 — DDL no MySQL: tipos, restrições e ordem de criação (30 min)

Resumo

DDL (*Data Definition Language*) cria e altera a **estrutura**: CREATE, ALTER, DROP.

Tipos que mais caem:

Conteúdo	Tipo	Por quê
identificador	BIGINT UNSIGNED AUTO_INCREMENT	inteiro, nunca negativo, gerado pelo banco
dinheiro	DECIMAL (10, 2)	exato ; FLOAT tem imprecisão e o caixa não fecha
código (CPF, CEP, telefone, placa)	CHAR(n) se o tamanho é fixo, VARCHAR(n) se varia	tem zeros à esquerda e nunca entra em conta
nome, descrição curta	VARCHAR(n)	tamanho variável
data	DATE, escrita 'AAAA-MM-DD'	
data e hora do evento	DATETIME	
quantidade	SMALLINT UNSIGNED ou INT	

Restrições — cada uma responde a uma pergunta do negócio:

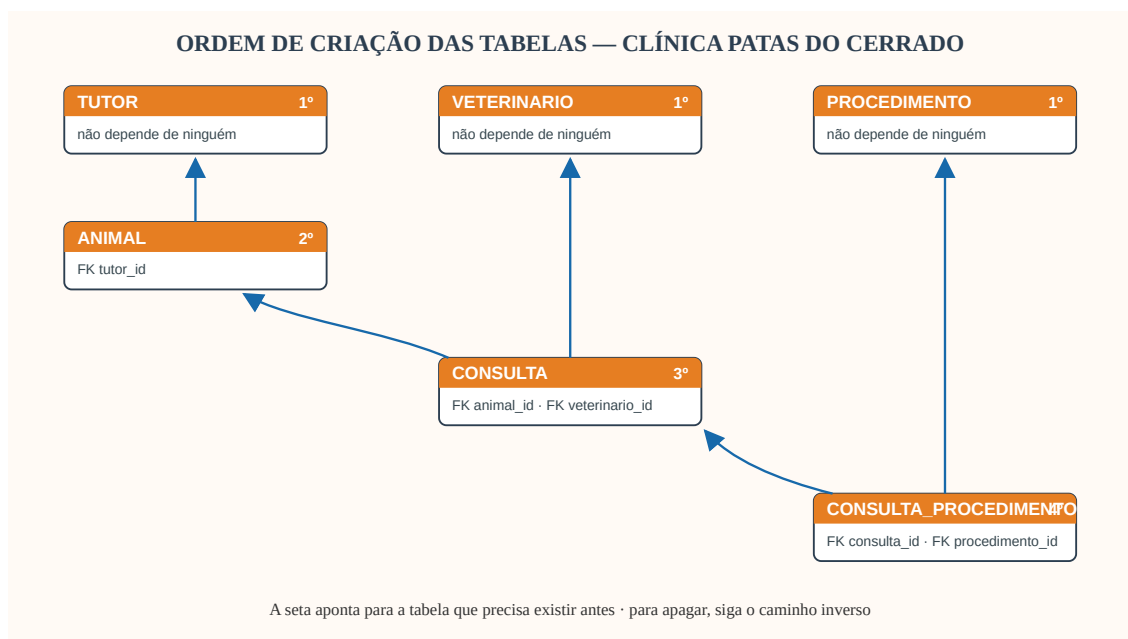
Restrição	Pergunta	Detalhe
NOT NULL	pode faltar?	sem ela, a coluna é opcional
DEFAULT	que valor usar quando não informado?	DEFAULT 'pendente'
PRIMARY KEY	o que identifica a linha?	nunca aceita NULL; uma por tabela
UNIQUE	pode repetir?	aceita NULL; usada na chave natural
CHECK	que valores fazem sentido?	aplicado de verdade desde o MySQL 8.0.16
FOREIGN KEY	depende de qual linha de outra tabela?	tipos iguais , inclusive UNSIGNED; a tabela pai precisa já existir

ON DELETE e ON UPDATE — o que acontece com os filhos quando o pai é apagado ou tem a chave alterada:

Ação	Efeito
RESTRICT (padrão)	impede apagar o pai enquanto houver filhos
CASCADE	repete a operação nos filhos: apaga junto
SET NULL	grava NULL na FK dos filhos — exige coluna opcional
NO ACTION	no MySQL, igual a RESTRICT

Padrão da disciplina: ON DELETE RESTRICT e ON UPDATE CASCADE.

Ordem de criação: primeiro as tabelas que **não dependem de ninguém**; depois as que têm FK. Para apagar, a ordem é a **inversa**.



Ordem de criação: primeiro quem não depende de ninguém

Exemplo resolvido — o script da corrida

Regras: *CPF obrigatório e sem repetição; percurso maior que zero; o número de peito não repete dentro da mesma prova; valor pago não negativo; a situação começa como ‘pendente’; não se apaga corredor nem prova que tenha inscrições.*

```

CREATE DATABASE IF NOT EXISTS corrida
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE corrida;

CREATE TABLE corredor (
  id      BIGINT UNSIGNED AUTO_INCREMENT,
  nome    VARCHAR(100)      NOT NULL,
  cpf     CHAR(11)          NOT NULL,
  cidade  VARCHAR(60),
  CONSTRAINT pk_corredor    PRIMARY KEY (id),
  CONSTRAINT uq_corredor_cpf UNIQUE (cpf)
);

CREATE TABLE prova (
  id      BIGINT UNSIGNED AUTO_INCREMENT,
  nome    VARCHAR(60)      NOT NULL,
  percurso_km DECIMAL(4,1) NOT NULL,
  data_prova DATE          NOT NULL,
  CONSTRAINT pk_prova      PRIMARY KEY (id),
  CONSTRAINT ck_prova_percurso CHECK (percurso_km > 0)
);

CREATE TABLE inscricao (
  id      BIGINT UNSIGNED AUTO_INCREMENT,
  corredor_id BIGINT UNSIGNED NOT NULL,
  prova_id  BIGINT UNSIGNED NOT NULL,
  numero_peito SMALLINT UNSIGNED NOT NULL,
  valor_pago DECIMAL(8,2)      NOT NULL,
  situacao  VARCHAR(20)        NOT NULL DEFAULT 'pendente',
  CONSTRAINT pk_inscricao PRIMARY KEY (id),
  CONSTRAINT fk_inscricao_corredor FOREIGN KEY (corredor_id)
    REFERENCES corredor (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT fk_inscricao_prova FOREIGN KEY (prova_id)
    REFERENCES prova (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT uq_inscricao_peito UNIQUE (prova_id, numero_peito),
  CONSTRAINT ck_inscricao_valor CHECK (valor_pago >= 0)
);

```

Repare: corredor e prova vêm primeiro (não dependem de ninguém); inscricao vem por último, porque tem as duas FKs. Os tipos das FKs (BIGINT UNSIGNED) são iguais aos das PKs que elas referenciam.

Ler um script e prever o erro

Todas as mensagens abaixo foram obtidas executando os comandos em um servidor MySQL 8.

O que o script faz	Resposta do MySQL
cria <code>inscricao</code> , com FK para <code>corredor</code> , antes de criar <code>corredor</code>	ERROR 1824 (HY000): Failed to open the referenced table 'corredor'
declara <code>corredor_id</code> INT apontando para <code>corredor.id</code> BIGINT UNSIGNED	ERROR 3780 (HY000): Referencing column 'corredor_id' and referenced column 'id' in foreign key constraint 'fk_inscricao_corredor' are incompatible.
declara <code>corredor_id</code> ... NOT NULL com ON DELETE SET NULL	ERROR 1830 (HY000): Column 'corredor_id' cannot be NOT NULL: needed in a foreign key constraint 'fk_inscricao_corredor' SET NULL
deixa uma vírgula antes do) final	ERROR 1064 (42000): You have an error in your SQL syntax; ... near ')' at line 4
executa o <code>CREATE TABLE corredor</code> uma segunda vez	ERROR 1050 (42S01): Table 'corredor' already exists
tenta <code>DROP TABLE corredor</code> com <code>inscricao</code> apontando para ela	ERROR 3730 (HY000): Cannot drop table 'corredor' referenced by a foreign key constraint 'fk_inscricao_corredor' on table 'inscricao'.
cria tabela sem ter executado USE	ERROR 1046 (3D000): No database selected

Depois de criadas as tabelas, as restrições barram os dados errados: 1452 para FK que aponta para pai inexistente, 1062 para valor repetido em UNIQUE, 3819 para CHECK violado e 1364 para coluna NOT NULL não informada.

Um comando que falha **não cria nada**: depois de corrigir a causa (por exemplo, criar a tabela pai), é preciso **executar de novo** o comando que falhou.

Armadilhas

- Dinheiro em FLOAT ou em INT. Dinheiro é DECIMAL.
- CPF, CEP ou telefone em INT: o zero à esquerda some.
- Achar que UNIQUE obriga a preencher. Quem obriga é o NOT NULL.
- FK com tipo diferente da PK (INT × BIGINT UNSIGNED).
- SET NULL em coluna NOT NULL.
- Achar que o MySQL “guarda a FK pendente” até a tabela pai aparecer. Ele recusa na hora.
- Escrever CHECK (`preco = 50`) quando a regra é “não pode ser negativo”: a restrição deve dizer exatamente a regra — CHECK (`preco >= 0`).

Perguntas de fixação

1. Escreva a coluna para a regra: “a nota fiscal tem valor total obrigatório, com centavos, que nunca é negativo”.
2. Um script cria, nesta ordem, pedido (com FK para cliente) e depois cliente. O que acontece em cada comando?
3. Em um sistema de prontuário, por que ON DELETE CASCADE na FK de consulta para paciente seria uma má escolha?

Confira a resposta

1. `valor_total DECIMAL(10,2) NOT NULL` e, entre as restrições, `CONSTRAINT ck_nota_valor CHECK (valor_total >= 0)`.
2. O primeiro falha com `ERROR 1824` (a tabela `cliente` ainda não existe) e nada é criado; o segundo cria `cliente`. Depois disso, é preciso executar de novo o `CREATE TABLE pedido`.
3. Porque apagar um paciente apagaria em silêncio todo o histórico de consultas dele. O `RESTRICT` protege o histórico: impede apagar o paciente enquanto houver consultas.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Etapa 10 — Simulado (40 min)

Agora é a hora de testar tudo junto, **nas mesmas condições da prova**:

1. Abra o arquivo de exercícios desta revisão (o **Simulado**).
2. Separe **40 minutos** sem interrupção. Guarde este roteiro, os slides e o caderno de estudo. **Sem consulta**, como na prova.
3. Responda as 10 objetivas anotando só a letra.
4. Escreva a discursiva **à mão, em papel**, com caneta, do começo ao fim — o DER desenhado e o `CREATE TABLE` escrito linha por linha. A prova é manuscrita: quem só treinou digitando descobre na hora que esquece vírgulas, parênteses e a ordem das palavras, e que desenhar o pé de galinha exige prática.
5. Só depois de terminar, passe para a **Resolução comentada do simulado**, logo abaixo, e corrija-se com o barema. Anote cada erro e volte à etapa indicada.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Resolução comentada do simulado

Faça o simulado antes de ler esta parte. Ler a resolução antes tira do simulado a única coisa que ele tem de valioso: mostrar o que você ainda não sabe.

Respostas das objetivas

Questão	1	2	3	4	5	6	7	8	9	10
Resposta	B	D	A	C	A	D	C	B	A	D

Conte seus acertos. Para cada erro, leia a explicação e volte à etapa indicada.

Questão 1 — B (Etapa 1)

I, II e III estão corretas. 42 sozinho é **dado**; a frase do boletim acrescenta onde, quando, o quê e a unidade: é **informação**; o alerta decidido a partir da regra da Defesa Civil é **conhecimento** aplicado.

A afirmação IV inverte os conceitos: anotar a mesma medição em dois cadernos é a **redundância**; os dois valores que não batem (42 e 24) são a **inconsistência**, que só aconteceu por causa da redundância.

- **A** e **C** incluem a IV, que é falsa.
- **D** deixa de fora a II, que é verdadeira.

Questão 2 — D (Etapa 3)

O desenho com retângulos, losangos e cardinalidades, sem tipos, mostrado ao dono, é o **conceitual**. A lista com PK e FK, sem tipos, que serve a qualquer SGBD, é o **lógico**. O CREATE TABLE com DECIMAL(10,2), escrito para o MySQL, é o **físico**.

- **A** troca conceitual e lógico: o desenho não fala em tabela nem em FK.
- **B** chama o documento 2 de físico, mas PK e FK já existem no nível lógico; físico é quando aparecem os tipos e o comando do produto.
- **C** chama o script de lógico. O SQL tem padrão, mas cada SGBD tem seu dialeto, e DECIMAL(10,2) NOT NULL dentro de CREATE TABLE é implementação.

Questão 3 — A (Etapa 4)

- **I verdadeira**: atleta tem 5 linhas de dados (tuplas) e 4 colunas (atributos).
- **II verdadeira**: a camisa 10 aparece em duas tuplas; chave primária não repete.
- **III falsa**: a FK **pode** repetir — os times 1 e 2 têm dois atletas cada. A integridade referencial exige que o valor **exista** em time.id, não que seja único.
- **IV verdadeira**: não há time de id 4, e o SGBD recusaria o atleta órfão.

Por isso, só **A** (I, II e IV) está correta.

Questão 4 — C (Etapa 5)

Perguntando nos dois sentidos: um palco recebe **vários** shows; cada show acontece em **um** palco → **1:N**, pé de galinha junto de SHOW. Um show reúne **várias** bandas; uma banda toca em **vários** shows → **N:M**, pé de galinha nas duas pontas; no lógico, **tabela associativa** (show_banda).

- **A** põe o pé de galinha junto de PALCO, o que diria que um show acontece em vários palcos, e resolve o N:M com uma coluna.
- **B** pergunta SHOW–BANDA em um sentido só e conclui 1:N.
- **D** faz PALCO–SHOW virar N:M, o que contradiz “cada show acontece em um único palco”.

Questão 5 — A (Etapa 6)

De dentro para fora: $\sigma_{\text{turno} = \text{'Noite'}}$ deixa as linhas 1, 4 e 5 (Violão, Bateria, Violão). $\pi_{\text{instrumento}}$ fica só com essa coluna e **elimina a repetição**: resultado com **1 coluna e 2 linhas**, Violão e Bateria.

- **B** esquece que a projeção elimina tuplas repetidas.
- **C** ignora a projeção: mantém as 4 colunas.
- **D** ignora a seleção: Teclado é da tarde.

Questão 6 — D (Etapa 7)

A asserção I é **verdadeira**: o erro de sintaxe (1064) é detectado pela **camada SQL** do servidor, na análise do comando, antes de qualquer gravação. A asserção II é **falsa**: o motor de armazenamento InnoDB não confere sintaxe; ele lê e grava os dados, com transações e chaves estrangeiras. Por isso: I verdadeira e II falsa.

Questão 7 — C (Etapa 8)

As duas asserções são verdadeiras e a II **justifica** a I. Um acontecimento que tem atributos próprios (data, número de cestas) e precisa ficar no histórico ganha identidade própria: vira **entidade de evento**, como a CONSULTA da clínica estudada nas aulas.

Questão 8 — B (Etapa 9)

CEP é **código**: tem 8 dígitos sempre, pode começar com zero e não entra em conta. O tipo certo é CHAR(8), e a regra “obrigatório” pede NOT NULL.

Conferido no MySQL 8: gravando '07700000' nas quatro opções, CHAR(8) guarda 07700000, enquanto INT e DECIMAL(8,0) guardam 7700000 — o zero da frente some.

- **A** e **C** perdem o zero à esquerda e tratam código como número.
- **D** não é obrigatória (falta NOT NULL) e ainda inventa um CEP '0' quando ninguém informa.

Questão 9 — A (Etapa 9)

O primeiro comando cria peca. O segundo pede ON DELETE SET NULL, que grava NULL em peca_id quando a peça é apagada — mas peca_id é NOT NULL. O MySQL recusa a tabela na hora:

```
ERROR 1830 (HY000): Column 'peca_id' cannot be NOT NULL: needed in a foreign key constraint 'fk_apresentacao_peca' SET NULL
```

Como a regra do grupo é preservar o histórico das apresentações, a correção é trocar por ON DELETE RESTRICT.

- **B** supõe que o MySQL aceita a combinação; ele não aceita.
- **C** descreve o erro 1824, mas peca já existe quando apresentacao é criada.

- **D** descreve o erro 3780, mas os dois tipos são BIGINT UNSIGNED.

Questão 10 — D (Etapa 2)

Cada peça doada tem campos diferentes conforme o tipo, a lista de tipos muda toda semana e não há regra ligando um registro a outro: é o caso típico que **pode favorecer um banco NoSQL de documentos**.

- **A** cria uma coluna para cada campo possível: a tabela fica cheia de colunas vazias e precisa de ALTER TABLE a cada tipo novo.
- **B** e **C** usam arquivo comum para dados compartilhados por várias voluntárias, com os problemas de concorrência e de inconsistência da Etapa 1.

Questão 11 — discursiva: resposta-modelo

a) DER com cardinalidades

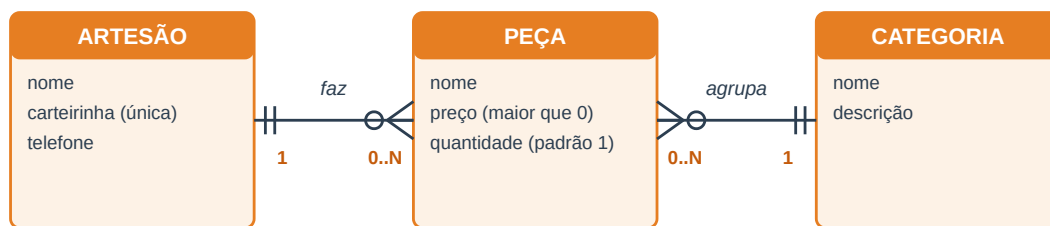
Entidades e atributos:

- **ARTESÃO**: nome, carteirinha (única), telefone;
- **CATEGORIA**: nome, descrição;
- **PEÇA**: nome, preço (maior que zero), quantidade (1 quando não informada).

Relacionamentos, perguntando nos dois sentidos:

- *Um artesão faz quantas peças? Várias (pode ainda não ter nenhuma). Uma peça é feita por quantos artesãos? Um.* → **ARTESÃO 1:N PEÇA**.
- *Uma categoria reúne quantas peças? Várias. Uma peça pertence a quantas categorias? Uma.* → **CATEGORIA 1:N PEÇA**.

DER — CATÁLOGO DE ARTESANATO (NOTAÇÃO PÉ DE GALINHA)



Um artesão faz 0..N peças; cada peça é de 1 artesão e de 1 categoria.

círculo = zero (opcional) · traço = um · pé de galinha = muitos

DER do catálogo de artesanato na notação pé de galinha

Junto de ARTESÃO e de CATEGORIA ficam os dois traços (exatamente um); junto de PEÇA, círculo e pé de galinha (zero ou muitos).

b) Modelo lógico

```
ARTESAO    (id PK, nome, carteirinha UQ, telefone)
CATEGORIA  (id PK, nome, descricao)
PECA       (id PK, artesao_id FK, categoria_id FK, nome, preco, quantidade)
```

As duas FKs ficam em peca, o lado N dos dois relacionamentos.

c) Script DDL, na ordem certa

```
CREATE DATABASE IF NOT EXISTS artesanato
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE artesanato;

CREATE TABLE artesao (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(100)    NOT NULL,
  carteirinha VARCHAR(10)     NOT NULL,
  telefone    VARCHAR(20),
  CONSTRAINT pk_artesao      PRIMARY KEY (id),
  CONSTRAINT uq_artesao_carteirinha UNIQUE (carteirinha)
);

CREATE TABLE categoria (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(60)     NOT NULL,
  descricao   VARCHAR(255),
  CONSTRAINT pk_categoria PRIMARY KEY (id)
);

CREATE TABLE peca (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  artesao_id  BIGINT UNSIGNED NOT NULL,
  categoria_id BIGINT UNSIGNED NOT NULL,
  nome        VARCHAR(100)    NOT NULL,
  preco       DECIMAL(10,2)   NOT NULL,
  quantidade  SMALLINT UNSIGNED NOT NULL DEFAULT 1,
  CONSTRAINT pk_pecas PRIMARY KEY (id),
  CONSTRAINT fk_pecas_artesao FOREIGN KEY (artesao_id)
    REFERENCES artesao (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT fk_pecas_categoria FOREIGN KEY (categoria_id)
    REFERENCES categoria (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT ck_pecas_preco CHECK (preco > 0)
);
```

As quatro regras do texto estão no script: carteirinha que não repete → UNIQUE; preço maior que zero → CHECK (preco > 0); quantidade 1 quando não informada → DEFAULT 1; não apagar artesanato com peças

→ FOREIGN KEY com ON DELETE RESTRICT.

artesanos e categorias podem ser criadas em qualquer ordem entre si, porque não dependem de ninguém; peça tem de vir por último. Se peça fosse criada primeiro, o MySQL responderia ERROR 1824 (HY000): Failed to open the referenced table 'artesanos'.

d) Apagar uma linha da tabela pai

A tabela pai é artesanos (e também categorias); a filha é peça. Quem decide o que acontece é a cláusula ON DELETE da chave estrangeira fk_peca_artesanos:

- com **RESTRICT** (a escolhida, porque a regra manda não apagar artesanos com peças), o MySQL **recusa** a exclusão enquanto houver alguma peça com aquele artesanos_id, e nada é apagado. A mensagem começa com ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails. Um artesanos sem nenhuma peça pode ser apagado normalmente;
- se fosse **CASCADE**, as peças do artesanos seriam apagadas junto, em silêncio;
- se fosse **SET NULL**, as peças ficariam com artesanos_id nulo — o que nem é aceito aqui, porque a coluna é NOT NULL.

É a **integridade referencial**: nenhuma peça pode ficar apontando para um artesanos que não existe.

Barema da discursiva (5,0 pontos)

Item	O que se espera	Pontos
a)	as três entidades com seus atributos	0,5
a)	as duas cardinalidades 1:N, justificadas nos dois sentidos	0,5
a)	DER desenhado no pé de galinha, com os símbolos do lado certo	0,5
b)	tabelas com id como PK, as duas FKs em peça e a carteirinha marcada como única	1,0
c)	os três CREATE TABLE na ordem certa, com tipos adequados (DECIMAL no preço), PK e FK de tipos iguais	1,0
c)	UNIQUE, CHECK, DEFAULT e NOT NULL nas colunas certas	1,0
d)	explica que o ON DELETE da FK decide; com RESTRICT, o MySQL recusa apagar o artesanos que tem peças	0,5
	Total	5,0

Descontos comuns: FK em artesanos (lado errado), preço em FLOAT, peça criada antes das tabelas pai, CHECK (preço >= 0) quando a regra diz “maior que zero”, e “o banco apaga tudo” como resposta do item d.

Ficou com dúvida? Escreva nos comentários do material no Classroom, dizendo a etapa e a pergunta.

Referências

CHEN, Peter Pin-Shan. The entity-relationship model: toward a unified view of data. **ACM Transactions on Database Systems**, New York, v. 1, n. 1, p. 9-36, mar. 1976.

CODD, Edgar F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, jun. 1970.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson, 2010.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. **Projeto de banco de dados: uma visão prática**. 17. ed. São Paulo: Érica, 2012.

ORACLE CORPORATION. **MySQL 8.4 reference manual**. Redwood City: Oracle, 2026. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/>. Acesso em: 27 set. 2026.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de gerenciamento de bancos de dados**. 3. ed. Porto Alegre: Grupo A, 2008.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020.